CS211 Group Project Report

2048 (5x5) and Luhn Checksum Validation

Session / group: MW Group01 (edit) *Members (GitHub usernames):* (edit - names and usernames only, no WSUIDs)
Repository: (edit)

1. Overview

The submission has two separate console programs written in standard C++17. **game2048** is a playable 5x5 2048 with scoring, win and game-over detection, and a single-slot save/load. **luhn** checks whether a 16-digit number passes the Luhn checksum. A passing checksum only shows the digits are internally consistent; it does not show that a card exists or is authorized.

2. Part A: 2048 design

Data. The board is a 5x5 `int` array (0 = empty, otherwise a power of two). `GameState` bundles the board, the score and the successful-move count. `MoveResult` carries `{changed, points}` back from each movement function.

Function	Purpose
slide_line(line, n, points)	Core of every move. Packs nonzero tiles toward index 0, scans left to right merging equal neighbours (skipping the partner so a tile merges at most once), repacks, and reports whether the line changed and the points earned.
move_left / move_right / move_up / move_down	Apply slide_line to every row or column. Right and down copy the line in reverse, slide it, and copy it back. They return a MoveResult and never insert tiles or touch the score, so they can be tested deterministically.
insert_random	Builds the list of empty cells and puts one 2 in a uniformly chosen one. Returns false on a full board. Called only by main, after a move that changed the board.
new_game	Clears the board, zeroes score and moves, and places two 2s in different empty cells.
is_game_over, has_empty_cell, highest_tile	Game over means no empty cell and no equal horizontal or vertical neighbours; a full board that still has a merge is not over.
save_game / load_game	Text file `savegame.txt`: marker CS211_2048_V1, then score and moves, then 5 rows of 5 integers. load_game checks the marker, two nonnegative int counters, exactly 25 cells that are 0 or powers of two (at least 2), int range, and no extra content. It fills temporaries and copies into the game only after the whole file passes.
main (game loop)	Reads one line per command (w a s d, v save, o load, n new game with y/n confirmation, h help, e exit), rejects anything else, and exits cleanly on end of input. On a changed move it adds the points, counts one move, inserts one 2, redisplays, and announces a win when the highest tile first reaches 2048 or more. Play continues after a win; at game over only v, o, n, h and e work.

Design choices.

- Movement is kept separate from random insertion, score and move counting, so the same board always gives the same result when testing.
- One shared `slide_line` handles all four directions, so the merge-once rule exists in exactly one place.
- An unchanged move does nothing: no tile, no score, no move count (the program says the move changes nothing).
- Loading is all-or-nothing: a missing or corrupt file leaves the current game exactly as it was and prints the reason.
- Controls are w/a/s/d for movement, with save on v because `s` is down.

3. 2048 manual dry run: one left move

The move is **left** on the board below ('.' is an empty cell). Each row is handled by slide_line in two steps: **pack** the nonzero tiles to the left, then **scan** left to right merging equal neighbours. After a pair merges, the scan skips the partner tile, so the new tile is never compared again.

2	2	2	.	.
4	4	8	8	.
2	.	.	4	2
2	2	4	.	.
2	4	8	.	.

Starting board

left move
-->

4	2	.	.	.
8	16	.	.	.
2	4	2	.	.
4	4	.	.	.
2	4	8	.	.

Final board before random insertion

Row	Start	After packing	Merge scan (left to right)	After merge	Points
0	2 2 2 . .	2 2 2	(2,2) merge to 4; skip partner. The last 2 has no partner left, so it stays.	4 2 . . .	+4
1	4 4 8 8 .	4 4 8 8	(4,4) merge to 8; skip partner. Then (8,8) merge to 16.	8 16 . . .	+24
2	2 . . 4 2	2 4 2	2 vs 4 differ; 4 vs 2 differ. No merges.	2 4 2 . .	+0
3	2 2 4 . .	2 2 4	(2,2) merge to 4; skip partner. The 4 has no partner left.	4 4 . . .	+4
4	2 4 8 . .	2 4 8	2, 4, 8 are all different. No merges.	2 4 8 . .	+0

- **Score gained:** 4 + 24 + 0 + 4 + 0 = **32**.
- **Did the board change?** Yes (rows 0 to 3 changed), so the move counts: moves + 1, and score + 32.
- **Should a new tile appear?** Yes. main then calls insert_random, which places exactly one **2** in a random one of the 13 empty cells. This earns no score. Had the board not changed, no tile would appear and nothing would be counted.

Why the merge-once rule holds. In row 0, 2 2 2 becomes 4 2, not 8: after the first two 2s merge into a 4, the scan skips past the partner, so the new 4 is never compared with the third 2. In row 1, 4 4 8 8 becomes 8 16, not 32: the 8 created by the 4s is not merged with the original 8s, which merge only with each other. In row 3, the 4 created from two 2s does not merge with the existing 4.

4. Part B: Luhn design

Function	Purpose
isValidFormat(number, message)	Input-format check only: exactly 16 characters, each 0-9. On failure it explains why (wrong length, or non-digit including spaces, letters and punctuation). Malformed input is therefore reported differently from a checksum failure.
checkLuhn(number, sum)	Walks the string from the rightmost digit to the leftmost, adding each transformed digit, and returns true when sum % 10 == 0. The input is passed by const reference and never modified.
luhnDigitValue(number, i)	Returns the transformed value of digit i: unchanged if it is an odd-numbered position from the right (the rightmost digit is position 0), otherwise doubled, subtracting 9 when the double exceeds 9 (same as adding its two digits).
main	Loop: read a line, quit on q/Q, reject malformed input with a message and ask again, otherwise print PASSES or FAILS with the sum. End of input exits cleanly.

The number is read as a string, so leading zeros are kept (`0000000000000018` passes with sum 10). The checksum is computed by our own code with no library or service.

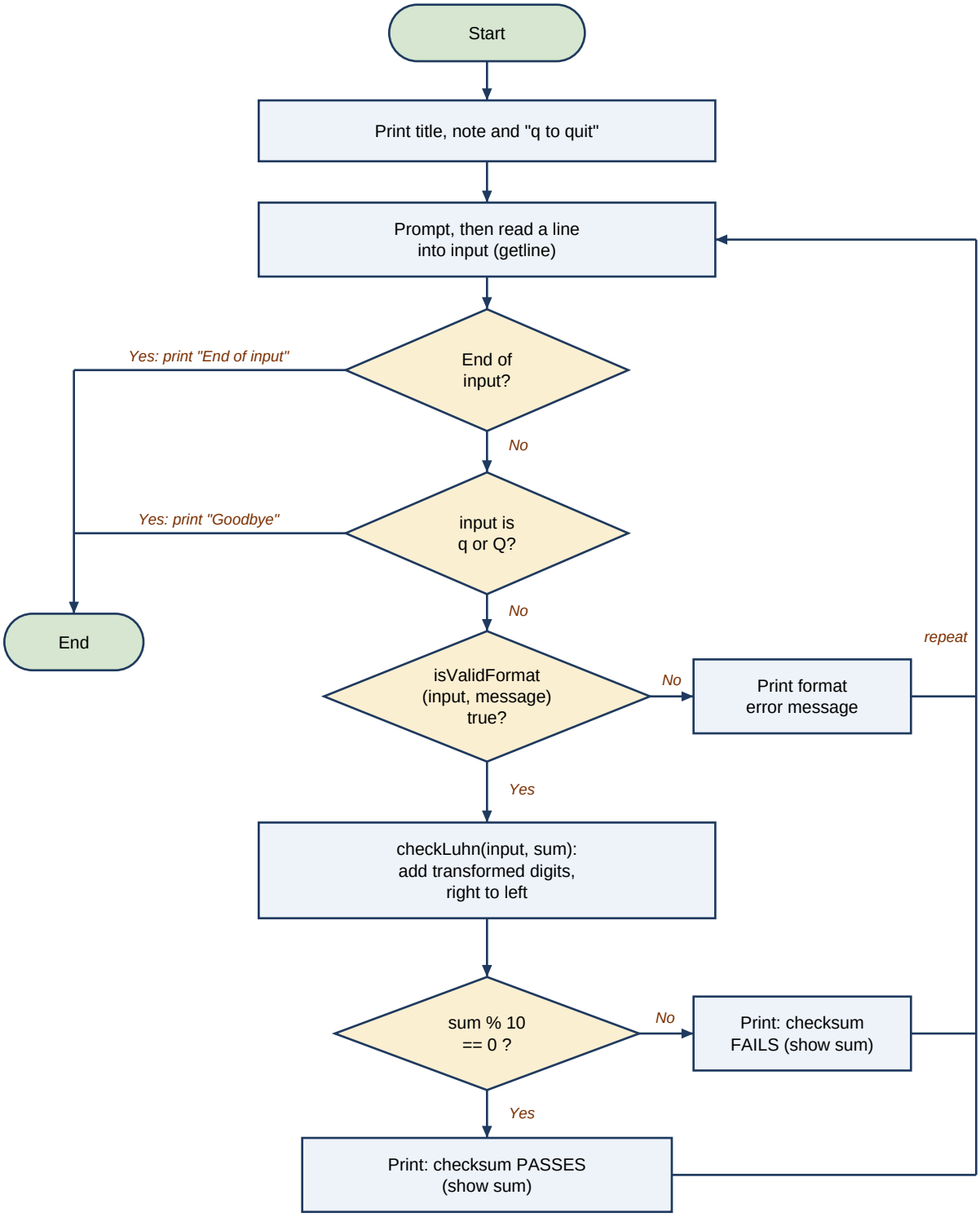
5. Luhn dry run: 4532015112830366

Traversal order: right to left. The rightmost digit is left unchanged, and every second digit moving left is doubled. A doubled value above 9 has 9 subtracted (equivalent to adding its two digits). All resulting values are added.

Step	Position from right	Original digit	Doubled?	Doubled value	Transformed value	Running sum
1	0	6	No	-	6	6
2	1	6	Yes	12	3	9
3	2	3	No	-	3	12
4	3	0	Yes	0	0	12
5	4	3	No	-	3	15
6	5	8	Yes	16	7	22
7	6	2	No	-	2	24
8	7	1	Yes	2	2	26
9	8	1	No	-	1	27
10	9	5	Yes	10	1	28
11	10	1	No	-	1	29
12	11	0	Yes	0	0	29
13	12	2	No	-	2	31
14	13	3	Yes	6	6	37
15	14	5	No	-	5	42
16	15	4	Yes	8	8	**50**

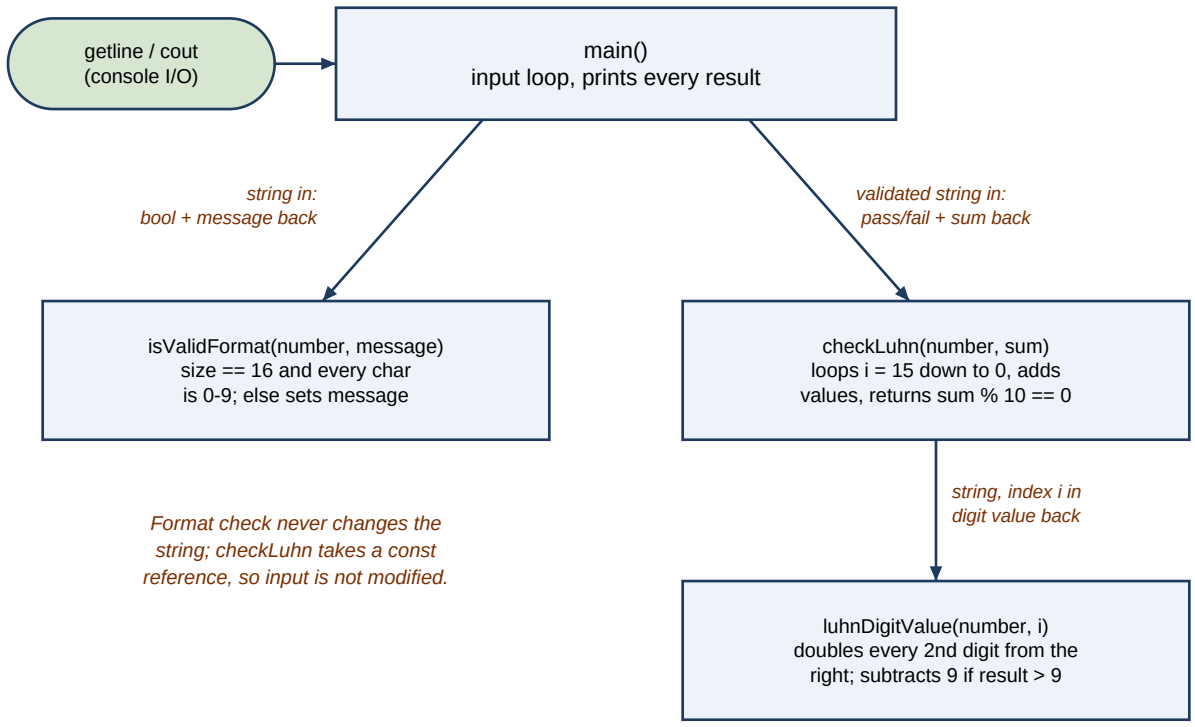
Final check: the sum is 50 and 50 % 10 = 0, so the sum is divisible by 10 and the number **passes** the Luhn checksum. The check digit is chosen so the whole sum lands on a multiple of 10, so a single wrong digit moves the sum off a multiple of 10 and is caught. **Contrast:** changing the last digit to 7 (4532015112830367) raises the first term from 6 to 7, giving a sum of 51; 51 % 10 = 1, so it fails. The program prints the same sums (50 and 51).

6. Luhn flowchart

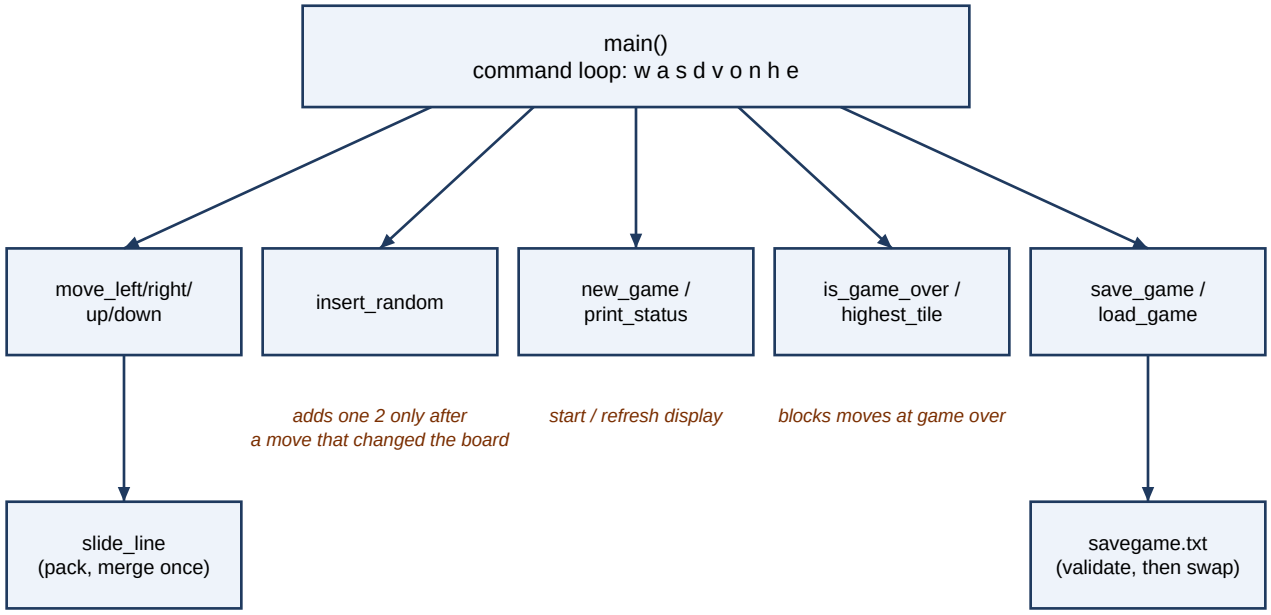


7. Function workflow diagrams

Luhn program: main, input validation, checkLuhn and its helper.



2048 program: how main uses the helper functions.



Source code and tests are in the project folders; see tests.md for expected and actual results and README.md for build and run commands.